

STAGE 8 — CH3: LINEAR SEARCH ALGORITHM

SLIDE 1 — From Idea to Algorithm

 **We know how Linear Search works...**

We already know:

 **Check each value one by one until the target is found.**

But how can we tell the **computer exactly what to do?** 

That's where an **algorithm** helps!

 **What is an Algorithm?**

An **algorithm** is a set of clear, step-by-step instructions used to solve a problem.

 **Think About It Like a Recipe**

 **Recipe:**

Take ingredients → Mix → Cook → Serve

 **Searching:**

Take data → Choose target → Compare → Move → Give result

 **Remember**

 **Algorithm = Step-by-step instructions**

SLIDE 2 — Linear Search Algorithm: The Steps

 Follow These Steps

Suppose:

Values: 10 25 40 15 30

Target: 15

Step 1 — Start

Begin with the **first element**.

 10

Step 2 — Compare

Compare the current element with the target.

10 == 15 

Step 3 — Move

If they don't match, move to the **next element**.

 25 → 40 → 15

Step 4 — Check Again

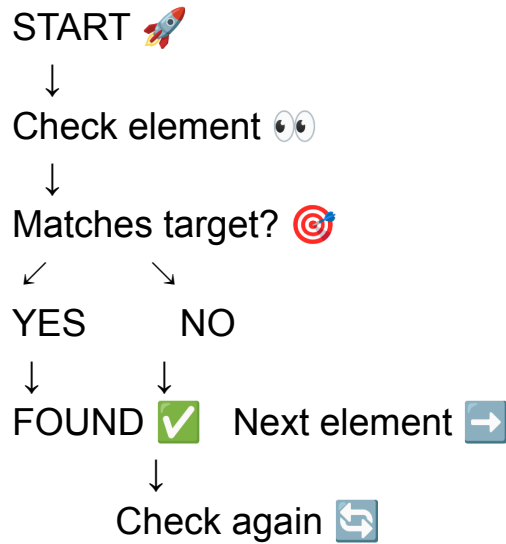
15 == 15 

 Target Found!

Step 5 — Stop

Once the target is found, the search can stop.

 Easy Flow



🟡 SLIDE 3 — Writing the Algorithm 🖋️

📋 Linear Search Algorithm

Let's write the steps in a form that is easy to follow:

Step 1: Start from the first element.

Step 2: Compare the current element with the target.

Step 3: If they match → **Target Found** ✅

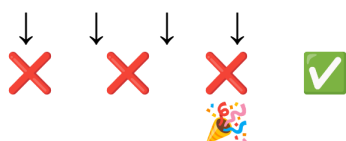
Step 4: If they don't match → move to the next element.

Step 5: Repeat until the target is found or all elements are checked.

Step 6: If all elements are checked without a match → **Not Found** ❌

🎯 Example

[10] [25] [40] [15] [30]



💡 Memory Trick

START → COMPARE → MOVE → REPEAT → RESULT

🟢 SLIDE 4 — Let's Trace the Algorithm 🔍

🔧 Dry Run

A **dry run** means following the algorithm manually to see what happens at each step.

Suppose:

Values = 8 12 20 25

Target = 20

🔍 Search Trace

Step	Current Value	Compare with 20	Result
1	8	$8 \neq 20$	Continue ➡
2	12	$12 \neq 20$	Continue ➡
3	20	$20 = 20$	Found ✅

👁️ Search Journey

8 ❌ → 12 ❌ → 20 ✅

🎉 The algorithm stops at 20.

🧠 Why Dry Run?

It helps us understand:

- ◆ Which values are checked
- ◆ Where the search stops
- ◆ Whether the algorithm is working correctly

 **Dry Run = Walk through the steps manually**

SLIDE 5 — Algorithm to C Code

Now let's turn our steps into C.

Algorithm

Start



Check current element



Compare with target



Match?



YES NO



Found



Next element



Repeat

C Code

```
int target = 20;
```

```
for(int i = 0; i < 4; i++) {  
    if(numbers[i] == target) {  
        printf("Found");  
        break;  
    }  
}
```

}

What Is Happening?

`i = 0`

 Start with the first element.

`numbers[i]`

 Gets the current value.

`== target`

 Checks whether it matches.

`break`

 Stops the search after finding it.

BIG MEMORY TRICK

Algorithm = Tell the computer WHAT to do, STEP BY STEP.

LEVEL CLEARED!

You can now:

 Understand an algorithm

 Follow Linear Search steps

 Trace a search

 Connect the steps to C code

LINEAR SEARCH ALGORITHM QUIZ

Q1. 💻 Fill in the Blank — Code

Complete the condition to compare the current element with the target.

```
if(numbers[i] == _____) {  
    printf("Found");  
    break;  
}
```

💡 **Hint:** Compare the current element with the value you are searching for.

📖 Glossary

- 🎯 **target** — The value being searched for.

✅ **Answer:** `target`

Q2. 💻 Fill in the Blank — Code

Complete the missing statement to stop the search after finding the target.

```
if(numbers[i] == target) {  
    printf("Found");  
    _____;  
}
```

💡 **Hint:** Use the statement that immediately stops the loop.

📖 Glossary

- 🔁 **loop** — A section of code that repeats.

-  **target** — The value being searched for.

✓ **Answer:** `break`

Q3. MCQ — Code

What will this code print?

```
int numbers[] = {10, 20, 30, 40};  
int target = 30;
```

```
for(int i = 0; i < 4; i++) {  
    if(numbers[i] == target) {  
        printf("Found");  
        break;  
    }  
}
```

- A) 10
- B) 20
- C) Found
- D) Not Found

 **Hint:** Check whether 30 appears in the values.

Glossary

-  **target** — The value being searched for.

✓ **Answer:** C) Found

Q4. Basic MCQ

What should happen when the current element **does not match** the target during Linear Search?

- A) Stop immediately
- B) Delete the element
- C) Move to the next element
- D) Sort the elements

 **Hint:** Linear Search checks the values one after another.

Glossary

-  **Linear Search** — A method of checking values one by one.
-  **target** — The value being searched for.

 **Answer: C) Move to the next element**

Q5. Code — Fill One Line

Complete the missing line so the search checks each element.

```
int numbers[] = {5, 10, 15, 20};
int target = 15;

for(int i = 0; i < 4; _____) {
    if(numbers[i] == target) {
        printf("Found");
        break;
    }
}
```

 **Hint:** After checking one element, **i** should move to the next position.

Glossary

-  **i** — The variable used to move through the positions.

-  **target** — The value being searched for.

✓ **Answer:** `i++`